Sri Lankan School Van Management System

Project Requirements Specification

1. System Overview

The School Van Management System is a comprehensive platform designed to streamline student transportation in Sri Lanka. It connects parents with reliable school van drivers, digitizes the seat booking process, and simplifies monthly fee tracking. The system features a mobile application with distinct driver and parent interfaces, managed by a centralized web portal for administrative oversight.

2. Technology Stack

- **Mobile Frontend (Driver & Parent):** Flutter (Cross-platform iOS and Android)
- **Backend API:** ASP.NET Core Web API
- **Database:** PostgreSQL (Relational data management for transactions and complex queries)
- **Admin Web Portal:** React / Next.js (For responsive and fast web accessibility)

3. User Roles and Modules

3.1 Driver Module (Mobile App)

- **Registration & Verification:** Drivers sign up and input personal details. They must upload photos of the van/bus, vehicle registration number, and specify the maximum passenger capacity.
- **Route & Pricing Configuration:** Drivers define their daily routes and the specific schools they cover. They can set fee structures (e.g., LKR per day, or a flat monthly rate per student).
- **Application Management:** When a parent applies for a seat, the driver receives a push notification. The driver can review the student's pickup location and school, and choose to **Accept** or **Decline** the request based on current bus capacity.
- **Payment & Bank Details:** Drivers input their local bank account details (Bank Name, Branch, Account Number). The app tracks which parents have paid for the month and who is pending.
- **Manual Payment Entry:** If a parent pays via cash in the van, the driver can manually log the payment to keep the system balanced.

3.2 Parent Module (Mobile App)

- **Parent Registration & Student Profiles:** Parents create an account and can add multiple student profiles. Each profile requires the student's name, school, and primary/emergency contact numbers.
- **Search & Discovery:** Parents search for available drivers based on their location, route, and the specific school.
- **Seat Application:** Parents send a seat request to a selected driver. Once approved by the driver, the finalized monthly/daily pricing is unlocked and displayed.
- **Payment Processing:**
  - **Bank Transfer:** The app displays the driver's bank details. Parents can perform a bank transfer and upload the receipt/slip as proof within the app.
  - **Cash Payment Logging:** Parents can mark a payment as "Paid via Cash", which sends a confirmation prompt to the driver.

3.3 Admin Portal (Web Portal)

- **Master Dashboard:** A high-level overview of total active drivers, registered students, and total vehicles in the system.
- **Driver Approval Workflow:** All newly registered drivers remain in a "Pending" state until an admin reviews their vehicle photos and details. Admins can **Approve** or **Reject** driver accessibility to ensure safety and quality control.
- **System Oversight:** Admins have read-only access to view all driver profiles, vehicle details, student lists assigned to specific vans, and general routing information.
- **Access Management:** Ability to block/suspend drivers or parents who violate terms.

4. Database Schema Requirements (High-Level)

- **Users Table:** Role-based access (Admin, Driver, Parent).
- **Vehicles Table:** Linked to Driver ID, stores capacity, photos, and plate numbers.
- **Students Table:** Linked to Parent ID, stores school name and emergency contacts.
- **Routes Table:** Defines the path and schools covered by a specific driver.
- **Applications Table:** Tracks seat requests (Status: Pending, Accepted, Declined).
- **Payments Table:** Tracks monthly transactions (Method: Cash/Bank, Status: Pending, Completed, Slip Image URL).

5. Development Environment Recommendations

For a streamlined local development experience, utilizing a `.vscode/tasks.json` configuration is highly recommended. This allows you to simultaneously spin up the ASP.NET Core backend API, the Next.js admin web portal, and the Flutter mobile emulator with a single command, ensuring seamless full-stack debugging and rapid iteration.